College of Electrical & Mechanical Engineering NUST

Department of Computer and Software Engineering

Name: Saad Mazhar Khan

NUST ID: 400692

Task: Assignment No.02

Submitted To: Dr. Usman Akram

Subject: Digital Image Engineering

Title: ***Image Enhancement and Analysis using Histogram, convolution and mathematical transformations***

DatasetLink:*https://drive.google.com/file/d/168xNxuDuYKT8FZTi_jHoWEZC4KQ9KZiJ/view?usp=sharing*

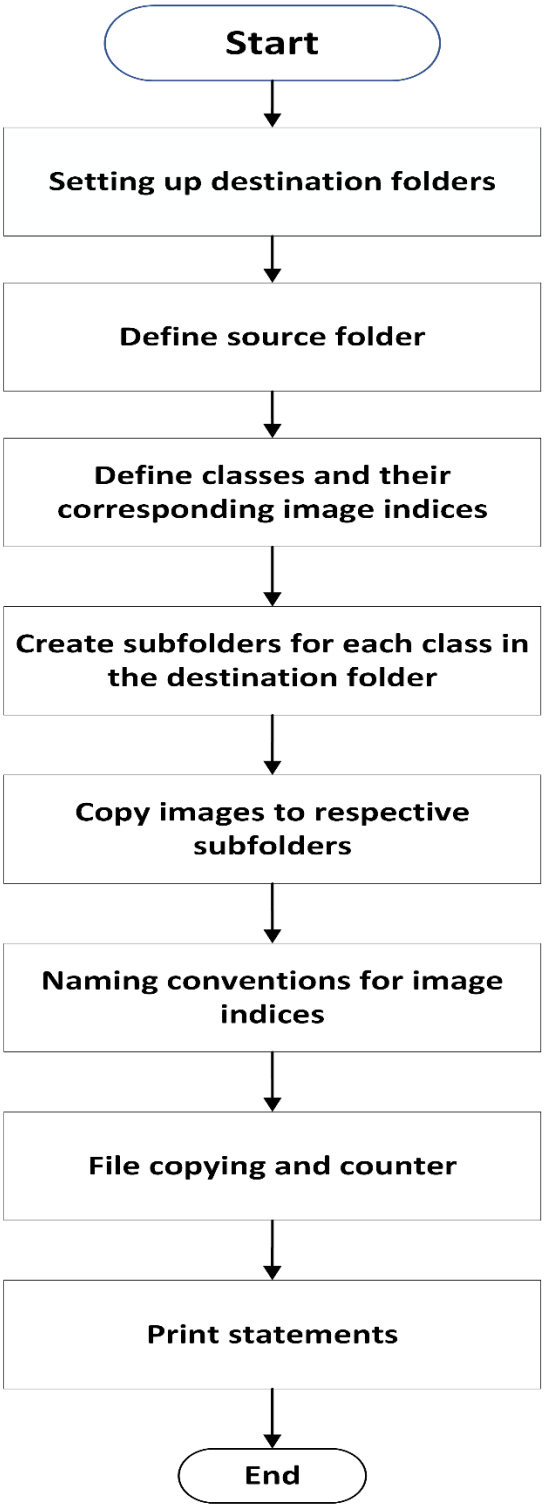
Date: 12-11-2023

Pre-Processing:

Before We starting Work on the given dataset, we split it in to three classes named as common nevi, atypical nevi, and melanoma. Steps are following what we do in this:

1. **Importing Libraries:**
    i. **os** and **shutil** libraries are imported. **os** is used for interacting with the operating system, and **shutil** is used for file operations.
2. **Setting Destination Folder:**
    i. **dest_add** is set to the destination address where the images will be copied.
3. **Creating Base Folder:**
    i. **Base_folder_name** is set to "Data_set".
    ii. **Base_folder_add** is created by joining **dest_add** and **Base_folder_name**.
    iii. If **Base_folder_add** doesn't exist, it is created using **os.makedirs**.
4. **Setting Source Folder:**
    i. **source** is set to the source address where the original images are located.
5. **Defining Class Dictionaries:**
    i. Three dictionaries (**Atypical**, **Common**, and **Mela**) are defined, each containing an 'address' and a list of 'index' values representing image numbers.
6. **Creating Subfolders and Copying Images:**
    i. A loop iterates over each class dictionary (**Atypical**, **Common**, **Mela**).
    ii. For each class, a subfolder (**sub_fol**) is created within the **Base_folder_add**.
    iii. Another loop iterates over the 'index' values of the class.
    iv. For each index, it checks if the image file exists in three possible formats (0-9, 10-99, 100-999) and copies it to the corresponding subfolder with a new name ("image{counter}.jpg").
    v. The counter is incremented for each successfully copied image.
7. **Print Statements:**
    i. Print statements are used to display messages indicating the successful copying of each image file.

Flow Diagram:



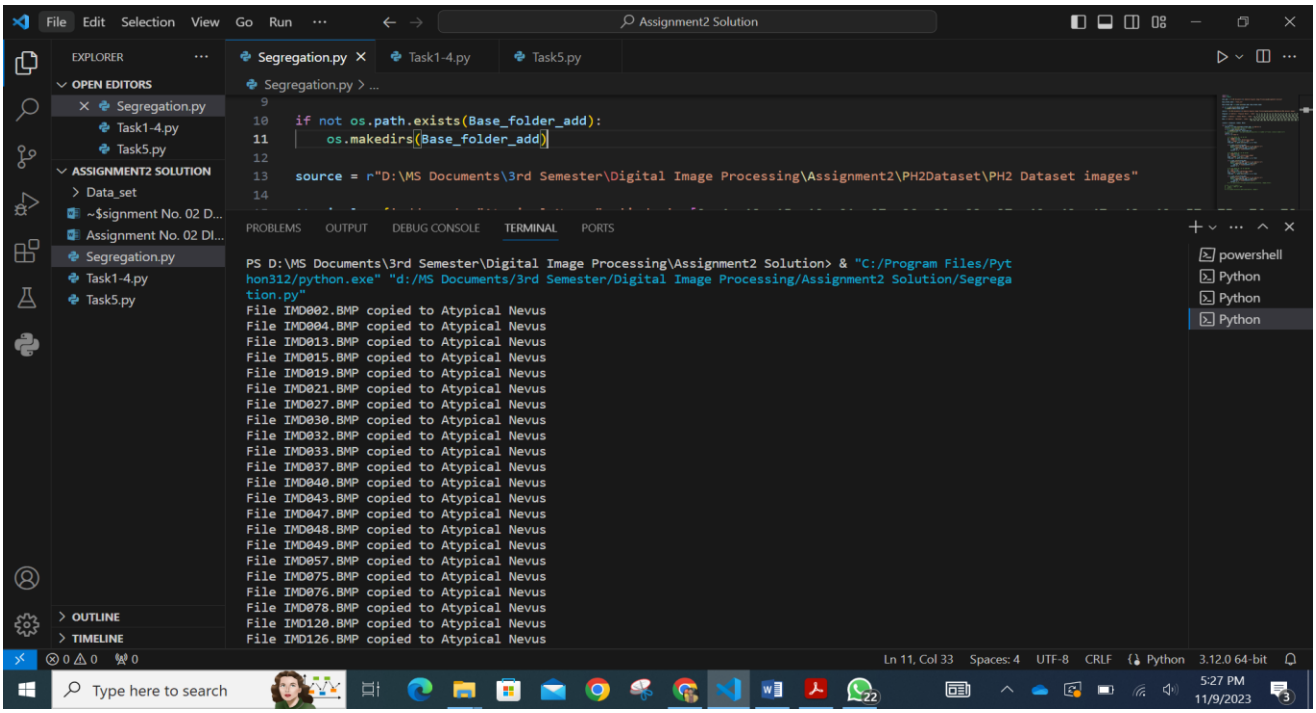
Code for this Task:

```
import os
import shutil

dest_add = r"D:\MS Documents\3rd Semester\Digital Image Processing\Assignment2 Solution"

Base_folder_name = r"Data_set"

Base_folder_add = os.path.join(dest_add, Base_folder_name)

if not os.path.exists(Base_folder_add):
    os.makedirs(Base_folder_add)

source = r"D:\MS Documents\3rd Semester\Digital Image Processing\Assignment2\PH2Dataset\PH2
Dataset images"

Atypical = {'address': "Atypical Nevus", 'index': [2, 4, 13, 15, 19, 21, 27, 30, 32, 33, 37,
40, 43, 47, 48, 49, 57, 75, 76, 78, 120, 126, 137, 138, 139, 149, 153, 157, 164, 166, 169,
171, 210, 347, 155, 376,
                                                    6, 8, 14, 18, 23, 31, 36, 154, 170, 226,
243, 251, 254, 256, 278, 279, 280, 304, 305, 306, 312, 328, 331, 339, 356, 360, 368, 369, 370,
382, 386, 388, 393, 396, 398, 427, 430, 431, 432, 433, 434, 436, 437]}
Common = {'address': "Common Nevus", 'index': [3, 9, 16, 22, 24, 25, 35, 38, 42, 44, 45, 50,
92, 101, 103, 112, 118, 125, 132, 134, 135, 144, 146, 147, 150, 152, 156, 159, 161, 162, 175,
177, 162, 198, 200, 10, 17,
                                                 20, 39, 41, 105, 107, 108, 133, 142, 143, 160,
173, 176, 196, 197, 199, 203, 204, 206, 207, 208, 364, 365, 367, 371, 372, 375, 374, 378, 379,
380, 381, 383, 385, 384, 389, 390, 392, 394, 395, 397, 399, 400, 402]}
Mela = {'address': "Melanoma", 'index': [58, 61, 63, 64, 65, 80, 85, 88, 90, 91, 168, 211,
219, 240, 242, 284, 285,
                                            348, 349, 403, 404, 405, 407, 408, 409, 410, 413,
417, 418, 419, 406, 411, 420, 421, 423, 424, 425, 426, 429, 435]}

classes = [Atypical, Common, Mela]

for i in classes:
    sub_fol = os.path.join(Base_folder_add, i['address'])
    # Ensure the subfolder exists or create it
    if not os.path.exists(sub_fol):
        os.makedirs(sub_fol, exist_ok=True)
    # print(f"subfolder : {os.path.abspath(sub_fol)} \n number of files: {len(i['index'])}")
    counter = 0
    for j in i['index']:
```

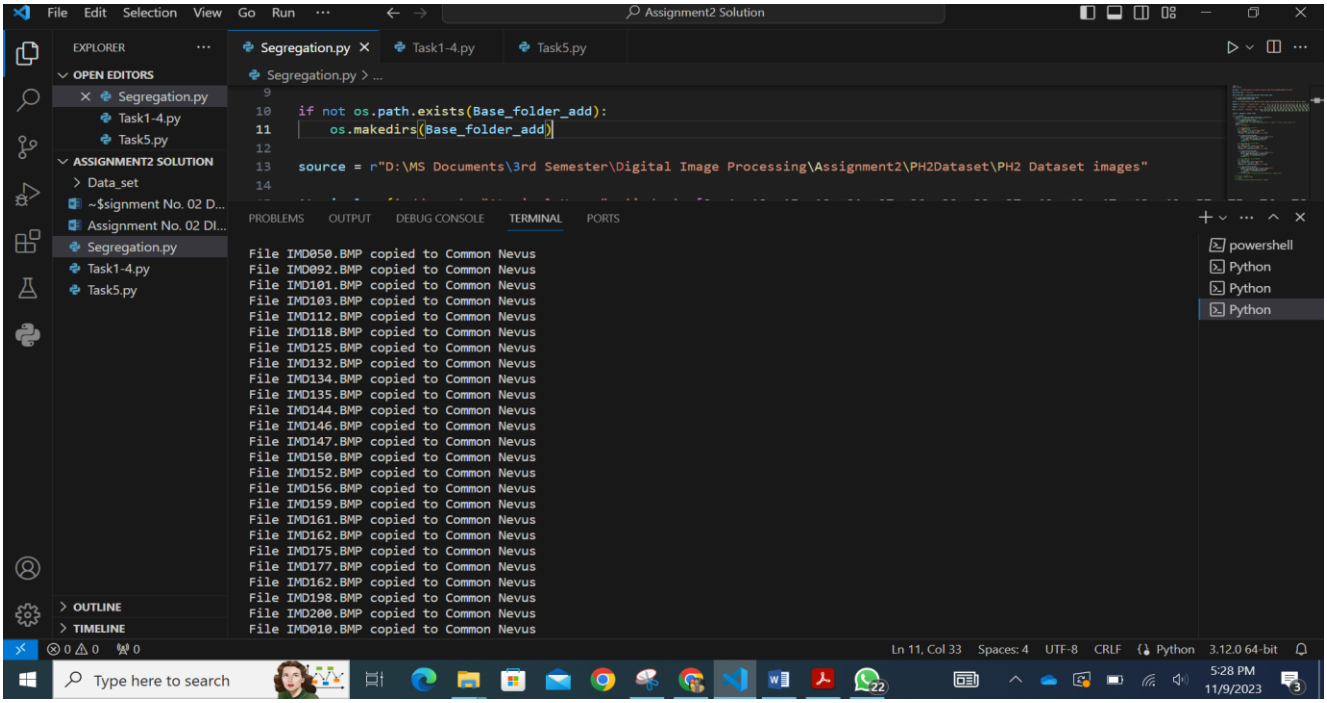
```
        # IF THE IMAGE NO. IS IN 0-9
        a = "IMD00"+str(j)
        b = "IMD00"+str(j)+"_Dermoscopic_Image"
        file = "IMD00" + str(j) + ".BMP"
        from_add = os.path.join(source, a, b, file)

        if os.path.isfile(from_add):
            print(f"File {a}.BMP copied to {i['address']}")
            shutil.copyfile(from_add, os.path.join(
                sub_fol, f'image{counter}.jpg'))
            counter += 1

        # IF THE IMAGE NO. IS IN 10-99
        a = "IMD0"+str(j)
        b = "IMD0"+str(j)+"_Dermoscopic_Image"
        file = "IMD0" + str(j) + ".BMP"
        from_add = os.path.join(source, a, b, file)

        if os.path.isfile(from_add):
            print(f"File {a}.BMP copied to {i['address']}")
            shutil.copyfile(from_add, os.path.join(
                sub_fol, f'image{counter}.jpg'))
            counter += 1

        # IF THE IMAGE NO. IS IN 100-999
        a = "IMD"+str(j)
        b = "IMD"+str(j)+"_Dermoscopic_Image"
        file = "IMD" + str(j) + ".BMP"
        from_add = os.path.join(source, a, b, file)

        if os.path.isfile(from_add):
            print(f"File {a}.BMP copied to {i['address']}")
            shutil.copyfile(from_add, os.path.join(
                sub_fol, f'image{counter}.jpg'))
            counter += 1
        # if os.path.exists(os.path.join(source_directory, image_file)):

    # for j in i['index']:
    #   image = 'IMD'+j+'.BMP'
    #  source =
    # if os.path.exists(os.path.join(source, image)):
```

Output for this Task:

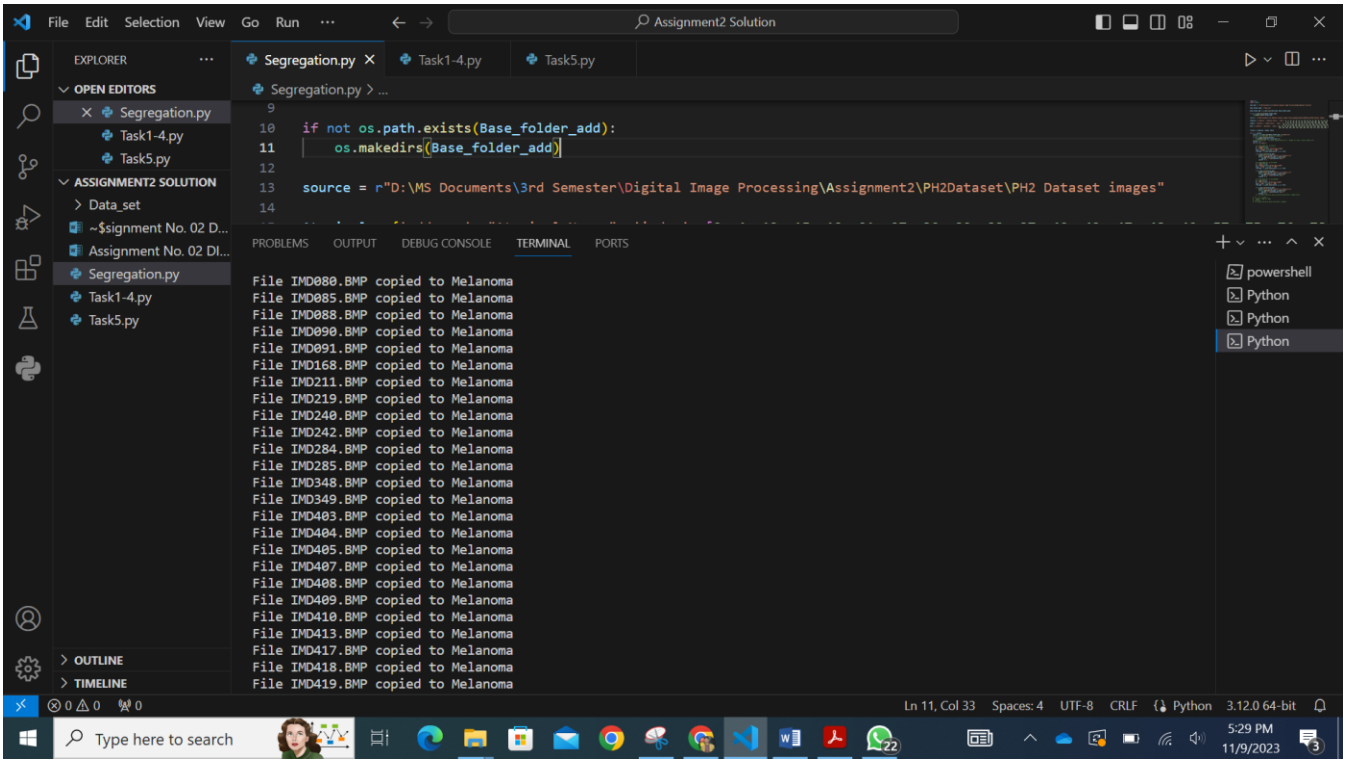
1:



2:



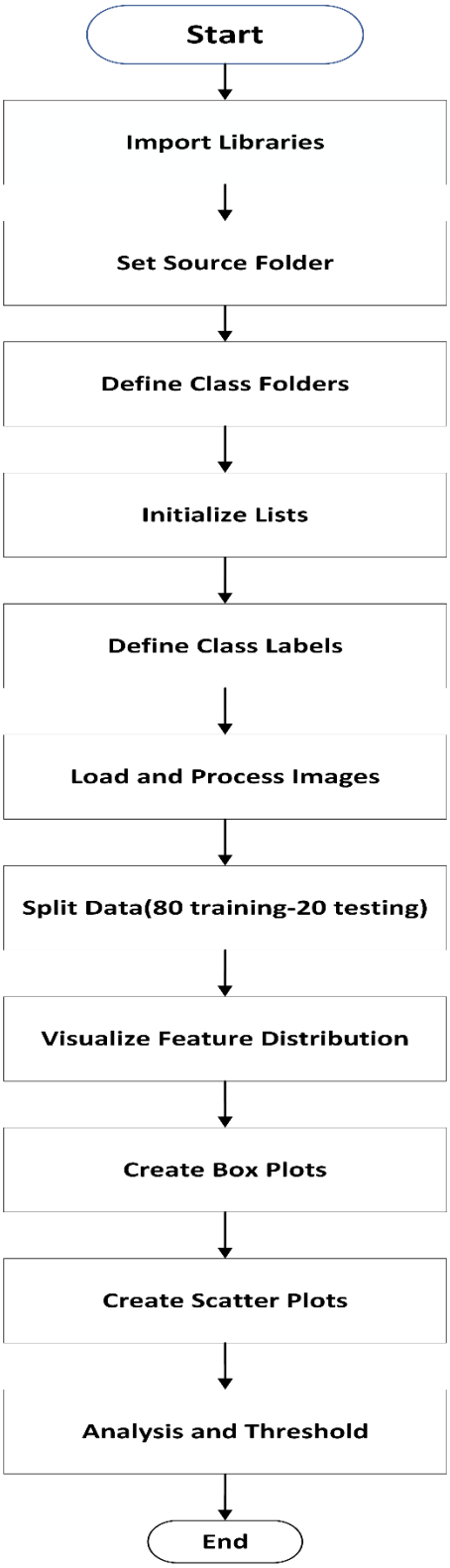
3:


Segregation.py
if not os.path.exists(Base_folder_add):
os.makedirs(Base_folder_add)
source = r"D:\MS Documents\3rd Semester\Digital Image Processing\Assignment2\PH2Dataset\PH2 Dataset images"
File IMD080.BMP copied to Melanoma
File IMD085.BMP copied to Melanoma
File IMD088.BMP copied to Melanoma
File IMD090.BMP copied to Melanoma
File IMD091.BMP copied to Melanoma
File IMD168.BMP copied to Melanoma
File IMD211.BMP copied to Melanoma
File IMD219.BMP copied to Melanoma
File IMD240.BMP copied to Melanoma
File IMD242.BMP copied to Melanoma
File IMD284.BMP copied to Melanoma
File IMD285.BMP copied to Melanoma
File IMD348.BMP copied to Melanoma
File IMD349.BMP copied to Melanoma
File IMD403.BMP copied to Melanoma
File IMD404.BMP copied to Melanoma
File IMD405.BMP copied to Melanoma
File IMD407.BMP copied to Melanoma
File IMD408.BMP copied to Melanoma
File IMD409.BMP copied to Melanoma
File IMD410.BMP copied to Melanoma
File IMD413.BMP copied to Melanoma
File IMD417.BMP copied to Melanoma
File IMD418.BMP copied to Melanoma
File IMD419.BMP copied to Melanoma

Task-01_04

1. Import Libraries:
    a. Import necessary libraries for image processing and machine learning.
2. Set Source and Class Folders:
    a. Define the source folder and paths for folders representing different classes of skin lesions.
3. Initialize Lists:
    a. Create empty lists to store image features and corresponding labels.
4. Define Class Labels:
    a. Assign numeric labels to each class of skin lesions.
5. Load Images and Extract Features (HOG):
    a. Loop through class folders, load images, resize, convert to grayscale, and compute Histogram of Oriented Gradients (HOG) features.
6. Split Data:
    a. Split the dataset into training and testing sets (80% training, 20% testing).
7. Visualize Feature Distribution (Box Plots):
    a. Choose a feature index, create box plots to visualize feature distribution for each class in the training set.
8. Visualize Feature Distribution (Scatter Plots):
    a. Create scatter plots to further visualize feature distribution by class in the training set.
9. Analysis and Thresholding:
    a. Visually inspect plots to determine optimal threshold values or conditions for feature segregation.
    b. Modify the code to apply specific thresholding or conditions based on the analysis.

Flow Diagram:



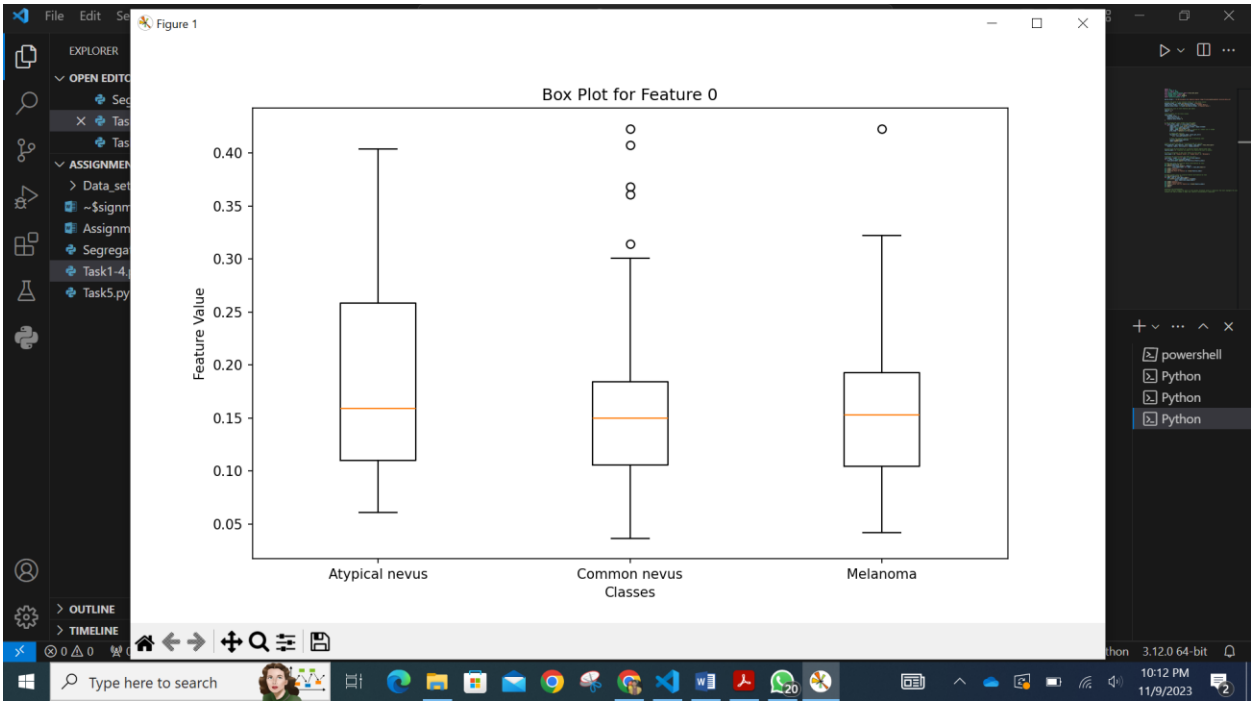
Code for Task 01-04:

```
import os
import numpy as np
from PIL import Image
from sklearn.model_selection import train_test_split
from skimage.feature import hog
from skimage.color import rgb2gray
import matplotlib.pyplot as plt

Source_folder = r"D:\MS Documents\3rd Semester\Digital Image
Processing\Assignment2 Solution\Data_set"

# Define the paths to your dataset folders for the three classes
melanoma_folder = os.path.join(Source_folder, f'Melanoma')
common_nevus_folder = os.path.join(Source_folder, f'Common Nevus')
atypical_nevus_folder = os.path.join(Source_folder, f'Atypical Nevus')

# Initialize lists to store features and labels
features = []
labels = []

# Define labels for the three classes
class_labels = {
    melanoma_folder: 2,
    common_nevus_folder: 1,
    atypical_nevus_folder: 0
}

# Loop through the class folders and load images
for class_folder, label in class_labels.items():
    for image_filename in os.listdir(class_folder):
        image_path = os.path.join(class_folder, image_filename)
        image = Image.open(image_path)
        image = image.resize((224, 224))  # Resize to a common size if needed
        gray_image = rgb2gray(np.array(image))

        # Compute HOG features
        hog_features = hog(gray_image, pixels_per_cell=(
            8,8), cells_per_block=(2, 2))

        # Store the computed features and corresponding label
        features.append(hog_features)
        labels.append(label)

# Split the data into 80% for training and 20% for testing
```

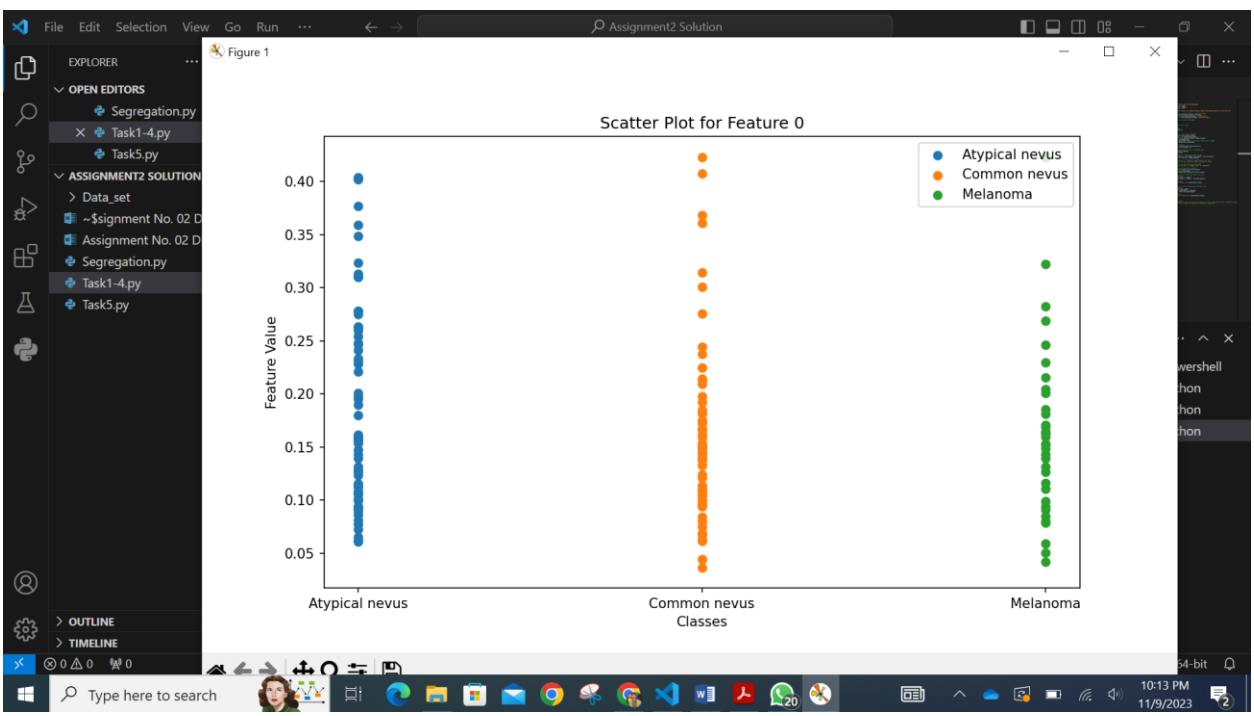
```python
train_features, test_features, train_labels, test_labels = train_test_split(
    features, labels, test_size=0.2, random_state=42)

# Visualizing the distribution of a specific feature against each class
feature_index = 0  # Specify the index of the feature you want to analyze

# Create a dictionary to map class labels to class names
class_names = {0: "Atypical nevus", 1: "Common nevus", 2: "Melanoma"}

# Separate the training data and labels by class
class_data = {label: [] for label in set(train_labels)}
for i, label in enumerate(train_labels):
    class_data[label].append(train_features[i][feature_index])

# Create box plots to visualize feature distribution by class
plt.figure(figsize=(10, 6))
plt.boxplot(class_data.values(), labels=[
            class_names[label] for label in class_data.keys()])
plt.xlabel('Classes')
plt.ylabel('Feature Value')
plt.title('Box Plot for Feature {}'.format(feature_index))
plt.show()

# Create scatter plots to visualize feature distribution by class
plt.figure(figsize=(10, 6))
for label, data in class_data.items():
    plt.scatter([class_names[label]] * len(data),
                data, label=class_names[label])
plt.xlabel('Classes')
plt.ylabel('Feature Value')
plt.title('Scatter Plot for Feature {}'.format(feature_index))
plt.legend()
plt.show()

# Analysis and Thresholding:
# You can visually inspect the plots to find optimal threshold values or
conditions that best segregate the cases.
# Modify the code as needed to apply your specific thresholding or conditions.
```

Output for this Code:

1: Boxplot:



2: Scatter Plot:



Task-05:

1. Import Necessary Libraries:
    a. Import required libraries, including os, numpy, PIL, scikit-learn modules, skimage, and pandas.

2. Define Source Folder and Class Paths:
    a. Specify the source folder and paths for different classes (Melanoma, Common Nevus, Atypical Nevus).

3. Initialize Feature and Label Lists:
    a. Create empty lists to store image features and corresponding labels.

4. Define Class Labels:
    a. Assign numerical labels to each class folder (0 for Atypical Nevus, 1 for Common Nevus, and 2 for Melanoma).

5. Load Images and Extract Features:
    a. Iterate through class folders, load images, resize them, convert to grayscale, and compute HOG features. Store features and labels in lists.

6. Split Data into Training and Testing Sets:
    a. Use the train_test_split function to split the data into 80% training and 20% testing sets.

7. Grid Search for Hyperparameters:
    a. Define a parameter grid for grid search, including values for C, kernel, and gamma.

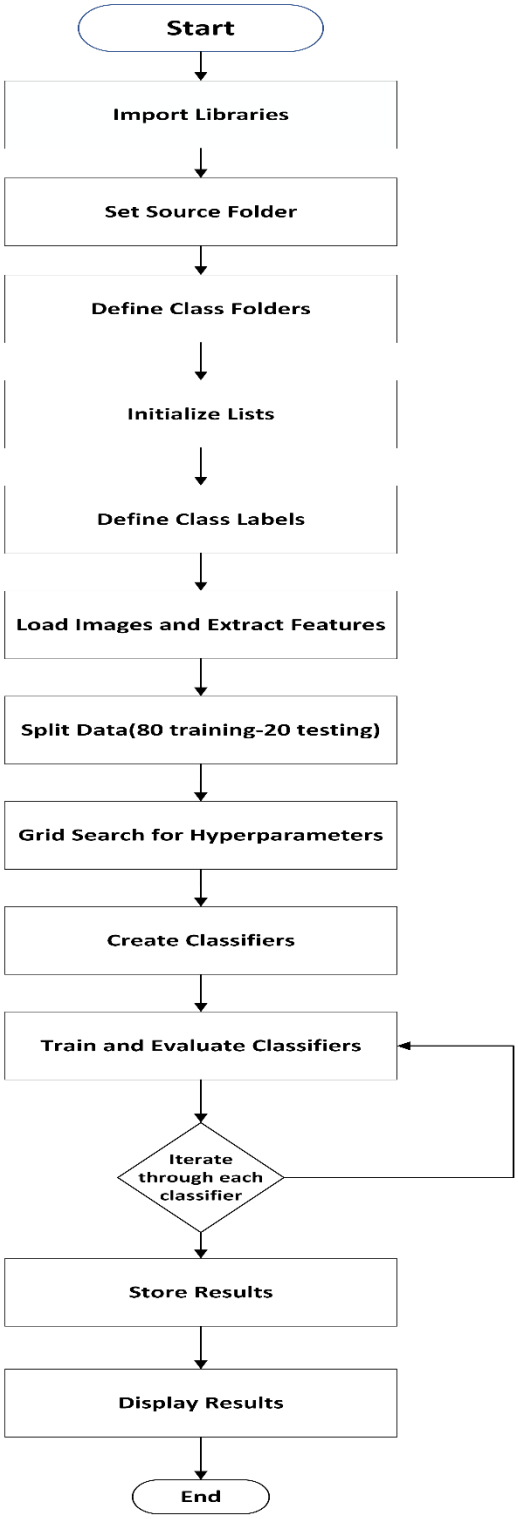
8. Create Classifiers:
    a. Instantiate KNeighborsClassifier, RandomForestClassifier, and a Support Vector Machine (SVC) with grid search for hyperparameters.

9. Train Classifiers and Evaluate Performance:
    a. Iterate through classifiers, train each on the training set, make predictions on the test set, calculate accuracy, and generate a classification report.

10. Create DataFrame for Results:
    a. Use pandas to create a DataFrame from the results obtained for each classifier.

11. Print Results Table:
    a. Use the tabulate library to print the results in a tabular format, including overall accuracy and accuracy per image for each classifier. Display the results table.

Flow Diagram:



Source Code for task 05:

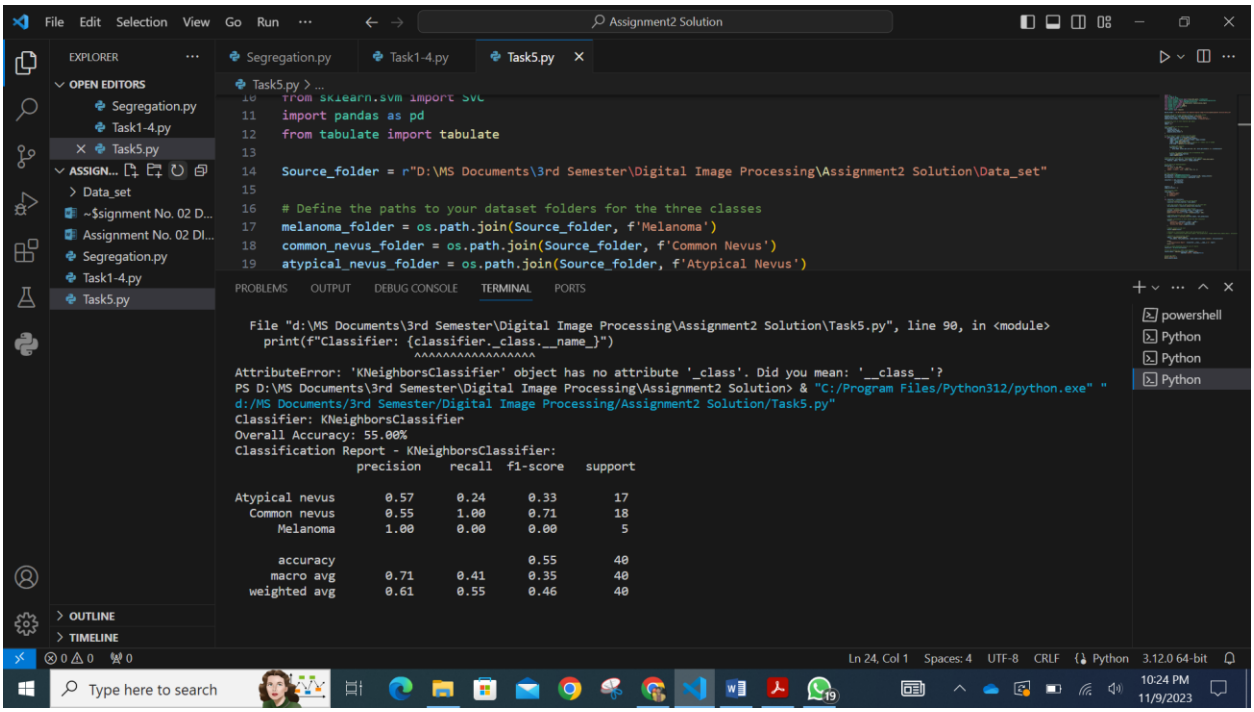
```
import os
import numpy as np
from PIL import Image
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report
from skimage.feature import hog
from skimage.color import rgb2gray
from sklearn.svm import SVC
import pandas as pd
from tabulate import tabulate

Source_folder = r"D:\MS Documents\3rd Semester\Digital Image
Processing\Assignment2 Solution\Data_set"

# Define the paths to your dataset folders for the three classes
melanoma_folder = os.path.join(Source_folder, f'Melanoma')
common_nevus_folder = os.path.join(Source_folder, f'Common Nevus')
atypical_nevus_folder = os.path.join(Source_folder, f'Atypical Nevus')

# Initialize lists to store features and labels
features = []
labels = []

# Define labels for the three classes
class_labels = {
    melanoma_folder: 2,
    common_nevus_folder: 1,
    atypical_nevus_folder: 0
}

# Loop through the class folders and load images
for class_folder, label in class_labels.items():
    for image_filename in os.listdir(class_folder):
        image_path = os.path.join(class_folder, image_filename)
        image = Image.open(image_path)
        image = image.resize((224, 224))  # Resize to a common size if needed
        gray_image = rgb2gray(np.array(image))

        # Compute HOG features
        hog_features = hog(
```

```
            gray_image, pixels_per_cell=(16, 16), cells_per_block=(2, 2),
orientations=9
        )

        # Store the computed features and corresponding label
        features.append(hog_features)
        labels.append(label)

# Split the data into 80% for training and 20% for testing
train_features, test_features, train_labels, test_labels = train_test_split(
    features, labels, test_size=0.2, random_state=42
)

# Grid Search for Hyperparameters
param_grid = {
    'C': [0.1, 1, 10, 100],
    'kernel': ['linear', 'rbf', 'poly'],
    'gamma': ['scale', 'auto', 0.001, 0.01, 0.1, 1]
}

# Create classifiers
knn_classifier = KNeighborsClassifier()
rf_classifier = RandomForestClassifier(n_estimators=100, random_state=42)
svm_classifier = GridSearchCV(SVC(), param_grid, cv=5)

classifiers = [knn_classifier,
               rf_classifier,
               svm_classifier
               ]

results = []
image_accuracies = []

# Define class names
class_names = {
    0: "Atypical nevus",
    1: "Common nevus",
    2: "Melanoma"
}

for classifier in classifiers:
    # Train the classifier using the training data
    classifier.fit(train_features, train_labels)

    # Use the trained model to make predictions on the test data
```

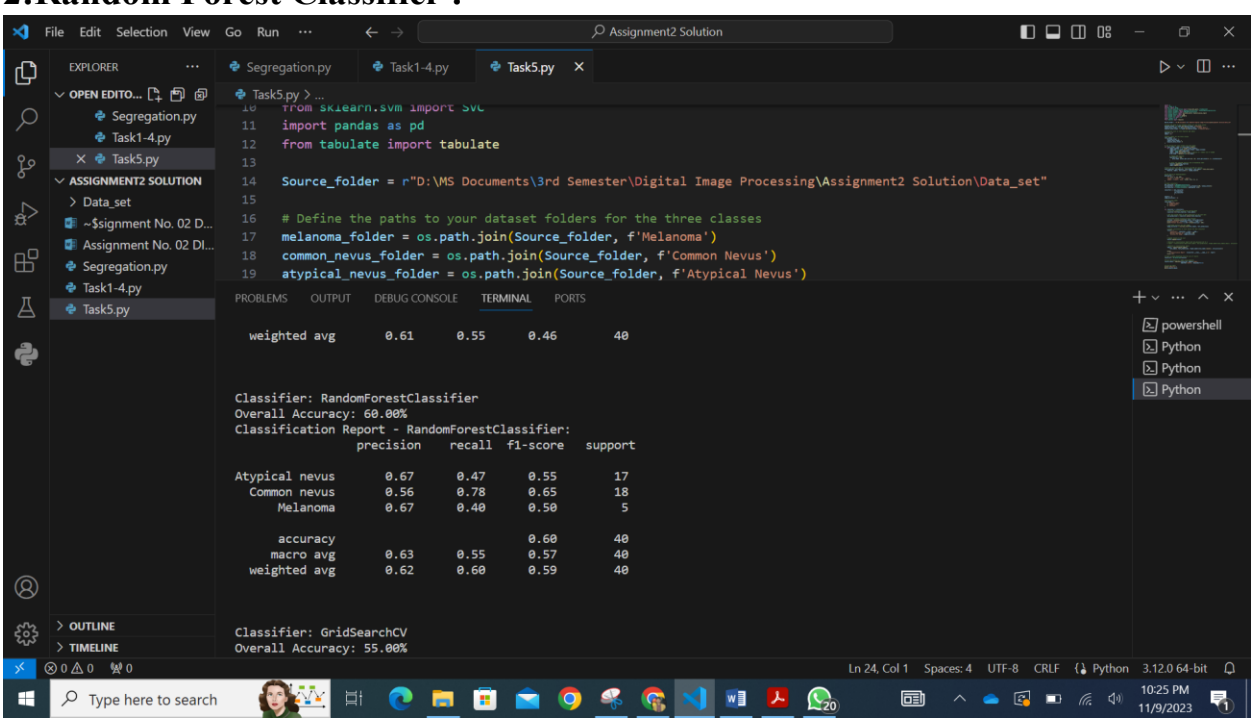
```
    test_predictions = classifier.predict(test_features)

    # Calculate overall accuracy of the solution
    accuracy = accuracy_score(test_labels, test_predictions)
    print(f"Classifier: {classifier.__class__.__name__}")
    print("Overall Accuracy: {:.2f}%".format(accuracy * 100))

    # Calculate accuracy for each image
    image_accuracies = accuracy_score(test_labels, test_predictions)

    # Create a dictionary to store results for this classifier
    result = {
        "Classifier": classifier.__class__.__name__,
        "Overall Accuracy (%)": accuracy * 100,
        "Accuracy Per Image": image_accuracies,
    }

    # Append results to the list
    results.append(result)

    # Generate a classification report with zero_division set to 1
    # report = classification_report(test_labels, test_predictions,
target_names=class_labels.keys(), zero_division=1)

    # Generate a classification report
    report = classification_report(
        test_labels, test_predictions, target_names=class_names.values(),
zero_division=1)

    print(
        f"Classification Report - {classifier.__class__.__name__}:\n", report)
    print("\n")

# Create a pandas DataFrame from the list of results
results_df = pd.DataFrame(results)

# Print the results as a table using tabulate
results_table = tabulate(results_df, headers='keys',
                         tablefmt='pretty', showindex=False)

# Print the table
print("\nResults:")
print(results_table)
```

Output:

1: KNN:



2:Random Forest Classifier :



3: Grid Search CV:

```
from sklearn.svm import SVC
import pandas as pd
from tabulate import tabulate

Source_folder = r"D:\MS Documents\3rd Semester\Digital Image Processing\Assignment2 Solution\Data_set"

# Define the paths to your dataset folders for the three classes
melanoma_folder = os.path.join(Source_folder, f'Melanoma')
common_nevus_folder = os.path.join(Source_folder, f'Common Nevus')
atypical_nevus_folder = os.path.join(Source_folder, f'Atypical Nevus')
```

```
  weighted avg       0.62      0.60      0.59        40


Classifier: GridSearchCV
Overall Accuracy: 55.00%
Classification Report - GridSearchCV:
                 precision    recall  f1-score   support

Atypical nevus       0.50      0.35      0.41        17
  Common nevus       0.58      0.78      0.67        18
      Melanoma       0.50      0.40      0.44         5

      accuracy                           0.55        40
     macro avg       0.53      0.51      0.51        40
  weighted avg       0.54      0.55      0.53        40


Results:
```

Overall Results:

```
from sklearn.svm import SVC
import pandas as pd
from tabulate import tabulate

Source_folder = r"D:\MS Documents\3rd Semester\Digital Image Processing\Assignment2 Solution\Data_set"

# Define the paths to your dataset folders for the three classes
melanoma_folder = os.path.join(Source_folder, f'Melanoma')
common_nevus_folder = os.path.join(Source_folder, f'Common Nevus')
atypical_nevus_folder = os.path.join(Source_folder, f'Atypical Nevus')
```

```
Atypical nevus       0.50      0.35      0.41        17
  Common nevus       0.58      0.78      0.67        18
      Melanoma       0.50      0.40      0.44         5

      accuracy                           0.55        40
     macro avg       0.53      0.51      0.51        40
  weighted avg       0.54      0.55      0.53        40


Results:
+-----------------------+---------------------+-------------------+
|      Classifier       | Overall Accuracy (%) | Accuracy Per Image |
+-----------------------+---------------------+-------------------+
|  KNeighborsClassifier |  55.00000000000001  |        0.55       |
| RandomForestClassifier |        60.0        |        0.6        |
|      GridSearchCV     |  55.00000000000001  |        0.55       |
+-----------------------+---------------------+-------------------+
PS D:\MS Documents\3rd Semester\Digital Image Processing\Assignment2 Solution>
```